

CH2

神經網路的數學概念

主講：邱偉豪

- [Google Colab Code](#)
- [原文：Deep Learning With Python](#)

1 初探神經網路

2 神經網路的資料表示法
張量 Tensor

3 神經網路的工具
張量運算

4 神經網路的引擎
以梯度為基礎的最佳化

5 重新檢視第一個例子

什麼是 MNIST?

深度學習的Hello World

核心定位

它是深度學習界的 HELLO WORLD

資料集介紹

- 包含 0~9 的手寫數字圖片。
- 訓練集： 60,000 張圖片與標籤（用於學習）。
- 測試集： 10,000 張圖片與標籤（用於考試）。
- 資料格式： 28X28 像素的灰階圖片。

```
from tensorflow.keras.datasets import mnist
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
```

載入MNIST資料集，自動分為訓練與測試兩組

train_images.shape

(60000, 28, 28)

test_images.shape

(10000, 28, 28)

可以用 .shape 檢查資料的結構

神經網路的骨架

```
from tensorflow import keras
from tensorflow.keras import layers
model = keras.Sequential([
    layers.Dense(512, activation="relu"),
    layers.Dense(10, activation="softmax")
])
```

模型的容器：Sequential

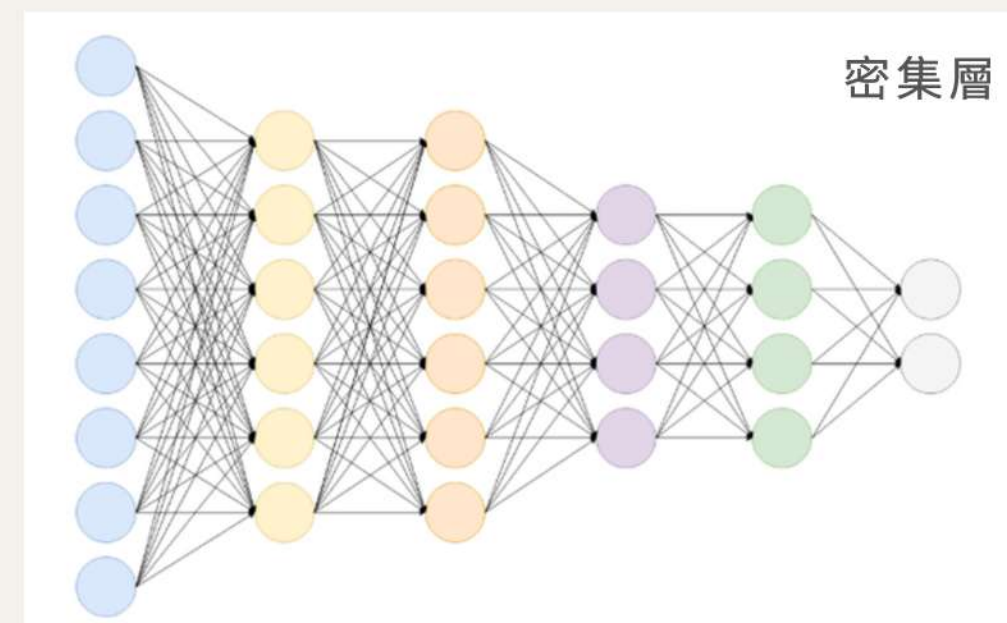
他是KERAS 中最簡單的模型定義方式

資料過濾器：層

- LAYERS.DENSE (密集層/全連接層)
 - 代表前後層的神經元兩兩相連，每個神經元都參與運算。
 - 第一層(512)的容量越大能學到的特徵越複雜，但也越耗資源
 - 最後一層對應輸出，也是我們的任務目標（0~9 共 10 個數字分類）

Softmax

將輸出的數值轉換為機率分佈，最後輸出 10 個機率值（加總為1）



學習的設定 & 資料預處理

準備圖片資料

```
train_images = train_images.reshape((60000, 28 * 28))
train_images = train_images.astype("float32") / 255
test_images = test_images.reshape((10000, 28 * 28))
test_images = test_images.astype("float32") / 255
```

```
model.compile(optimizer="rmsprop",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
```

編譯模型

Optimizer (優化器)

根據損失函數算出的錯誤分數，回頭去修改神經網路中的權重 (WEIGHTS)

Loss Function (損失函數)

衡量預測值 (PREDICTION) 與真實答案 (TRUE LABEL) 之間的距離

Metrics (評量指標)

我們人類看的指標 (如 ACCURACY 準確率)

在處理前我們要將資料變成介於[0,1]的區間，
處理前像素儲存於28,28的三軸陣列，處理後
會變成28*28的2軸向量陣列

astype('float32') / 255 (將數值縮放到 0-1)

訓練與結果

```
test_digits = test_images[0:10]
predictions = model.predict(test_digits)
predictions[0]
```

```
1/1 _____ 0s 72ms/step
array([9.1809076e-09, 9.5274251e-09, 4.1324197e-06, 1.6361028e-04,
       1.2448906e-11, 1.9005313e-08, 2.8378010e-12, 9.9983025e-01,
       4.9141132e-08, 1.9558631e-06], dtype=float32)
```

```
predictions[0].argmax() # 印出機率最高的數字->7
```

```
np.int64(7)
```

```
predictions[0][7]
```

```
np.float32(0.99983025)
```

```
test_labels[0] # 檢查正確答案 -> 7
```

```
np.uint8(7)
```

程式2.5 訓練神經網路模型

```
model.fit(train_images, train_labels, epochs=5, batch_size=128)
```

```
Epoch 1/5
469/469 _____ 6s 10ms/step - accuracy: 0.8743 - loss: 0.4381
Epoch 2/5
469/469 _____ 6s 12ms/step - accuracy: 0.9652 - loss: 0.1170
Epoch 3/5
469/469 _____ 5s 10ms/step - accuracy: 0.9795 - loss: 0.0708
Epoch 4/5
469/469 _____ 5s 10ms/step - accuracy: 0.9858 - loss: 0.0496
Epoch 5/5
469/469 _____ 6s 12ms/step - accuracy: 0.9890 - loss: 0.0355
<keras.src.callbacks.history.History at 0x7ef73a25fb30>
```

epochs=5：模型要把訓練集看過 5 遍

batch_size=128：看完128張後，統整錯誤再進行一次權重修正

469/469：60000/128 = 468.75

使用模型來進行預測

輸入： 我們丟給模型前 10 張沒看過的測試集圖片

輸出結果中9.998出現在第七個位置

代表模型斷定圖片數字為7

```
test_loss, test_acc = model.evaluate(test_images, test_labels) # 考試：用測試集打分數
print(f"test_acc: {test_acc}")
```

```
313/313 _____ 1s 4ms/step - accuracy: 0.9789 - loss: 0.0740
test_acc: 0.9812999963760376
```


什麼是張量 (Tensor)

Scalar Vector Matrix Tensor

1

$\begin{bmatrix} 1 \\ 2 \end{bmatrix}$

$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$

$\begin{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix} & \begin{bmatrix} 3 & 2 \end{bmatrix} \\ \begin{bmatrix} 1 & 7 \end{bmatrix} & \begin{bmatrix} 5 & 4 \end{bmatrix} \end{bmatrix}$

張量實際上看似很複雜，但其實它就和 list 一樣，也是存資料的容器，數學上可理解為多維數組。

0階張量為純量、1階張量為向量、2階張量為矩陣、3階張量則由多個矩陣所組成的。

“向量”和“張量”：

一個向量若有3個元素，會稱之為3維向量，而不是3D張量。

一3D向量只有一個軸，軸上有3個維度，而一3D張量有3個軸。

也可以將3D張量說為3階向量。



何為張量(Tensor)? 三分鐘圖解類神經網路基本資料結構

工欲善其事，必先利其器，若想做好人工智慧的研究，就必須對其資料結構的組成有一定的了解。本文會以圖文、程式參雜的方式講解張量的架構及其資料儲存的方式。

張量的分類

階數 (Rank / ndim): 軸的數量，你需要幾個索引才能選出一個具體的數字？

形狀 (Shape): 每個軸的維度大小，這個容器在每個方向上有多少個元素？

資料型別 (Dtype): 張量儲存的資料類型(float32、unit8、float64)，容器裡裝的是什麼材質？

```
from tensorflow.keras.datasets import mnist
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
```

```
train_images.ndim
print(train_images.ndim)
print(train_images.shape)
print(train_images.dtype)
```

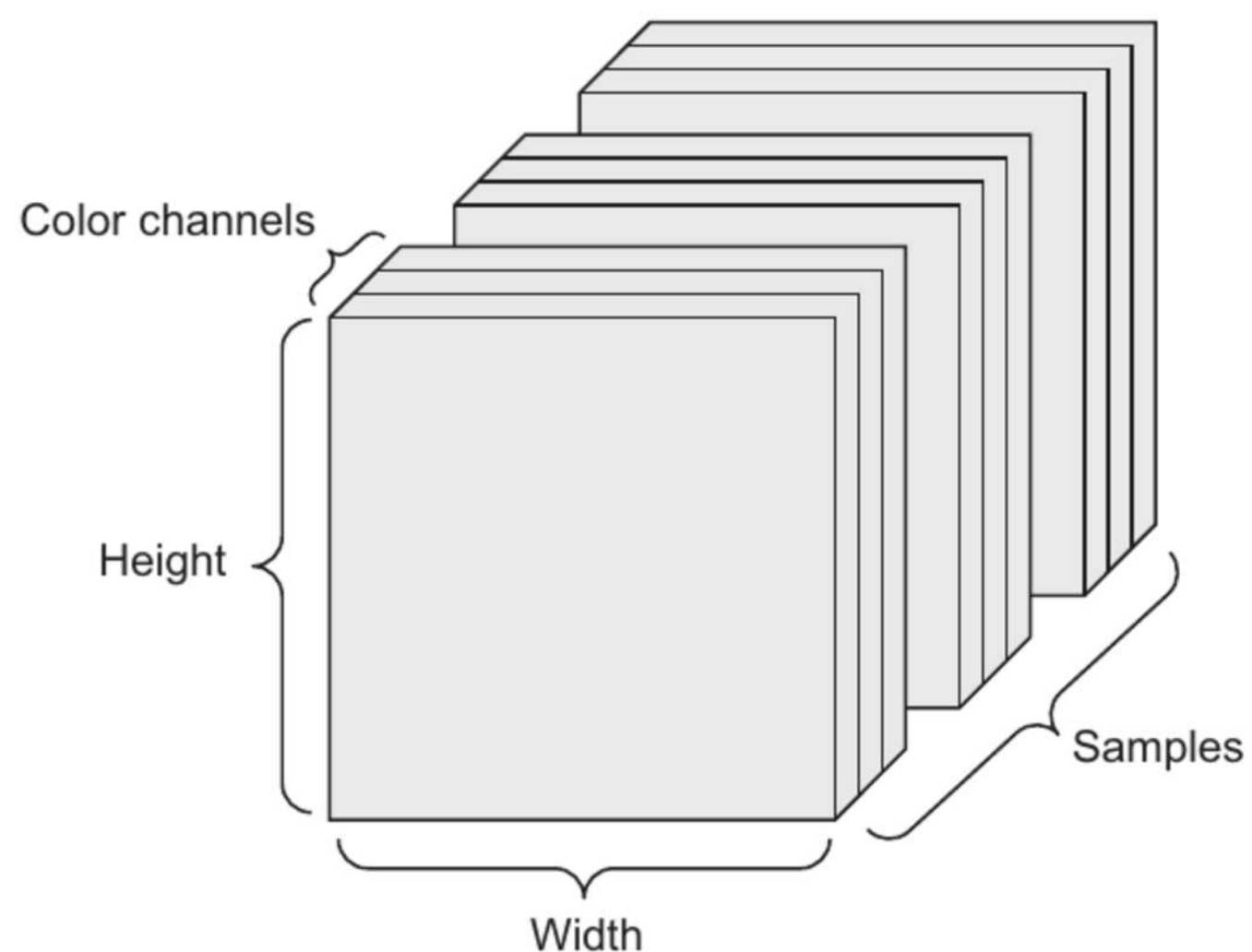
```
3
(60000, 28, 28)
uint8
```

```
x = np.array([[5, 78, 2, 34, 0],
              [6, 79, 3, 35, 1],
              [7, 80, 4, 36, 2]])
```

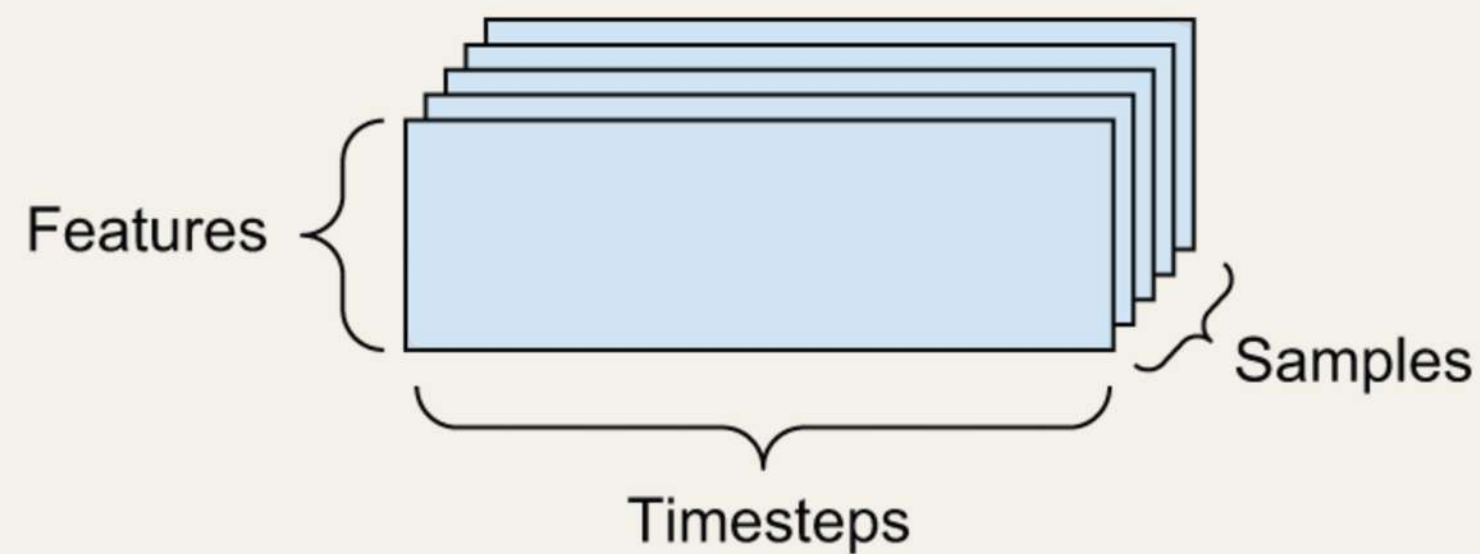
```
print(x.ndim)
print(x.shape)
print(x.dtype)
```

```
2
(3, 5)
int64
```


實務中的資料張量



- 向量資料：如人口統計資料。
 - (samples, features)
- 時間序列：如股票價格。
 - (samples, timesteps, features)
- 圖片資料：每個samples是由像素組成的2D方格(大小為 $h * w$)
 - (samples, height, width, channels)
- 影片資料：一序列的畫格
 - (samples, frames, height, width, channels)



逐元素運算

```
def naive_relu(x):  
    assert len(x.shape) == 2 # 不是2D就會Error  
    x = x.copy()  
    for i in range(x.shape[0]):  
        for j in range(x.shape[1]):  
            x[i, j] = max(x[i, j], 0)  
            #x[i, j] += y[i, j] 逐行相加也是同樣的執行方式  
    return x
```

```
import time  
# 建立兩個形狀為(20, 100)的2D張量  
x = np.random.random((20, 100))  
y = np.random.random((20, 100))  
# 使用NumPy執行逐元素運算  
t0 = time.time()  
for _ in range(1000):  
    z = x + y #逐元素相加  
    z = np.maximum(z, 0.)#逐元素relu運算  
print("Took: {0:.2f} s".format(time.time() - t0))
```

Took: 0.01 s

```
t0 = time.time()  
for _ in range(1000):  
    z = naive_add(x, y)  
    z = naive_relu(z)  
print("Took: {0:.2f} s".format(time.time() - t0))
```

Took: 3.06 s

只用基本python需要花上3.06秒的時間，因為用NumPy時他會將工作委託給BLAS執行，BLAS屬於底層平行化的高效率張量運算

張量擴張 (Broadcasting)

問題

當兩個張量形狀不同，怎麼相加？
(例：大矩陣 + 小向量)

規則

1. 小張量自動增加軸
2. 資料複製填滿新軸，直到形狀匹配

```
import numpy as np
X = np.random.random((32, 10))
y = np.random.random((10,))
y.shape
```

(10,)

```
y = np.expand_dims(y, axis=0) # y的sh
y.shape
```

在y中加入一個空的第0軸

(1, 10)

```
Y = np.concatenate([y] * 32, axis=0) #
Y.shape
```

在新軸重複放入y張量32次

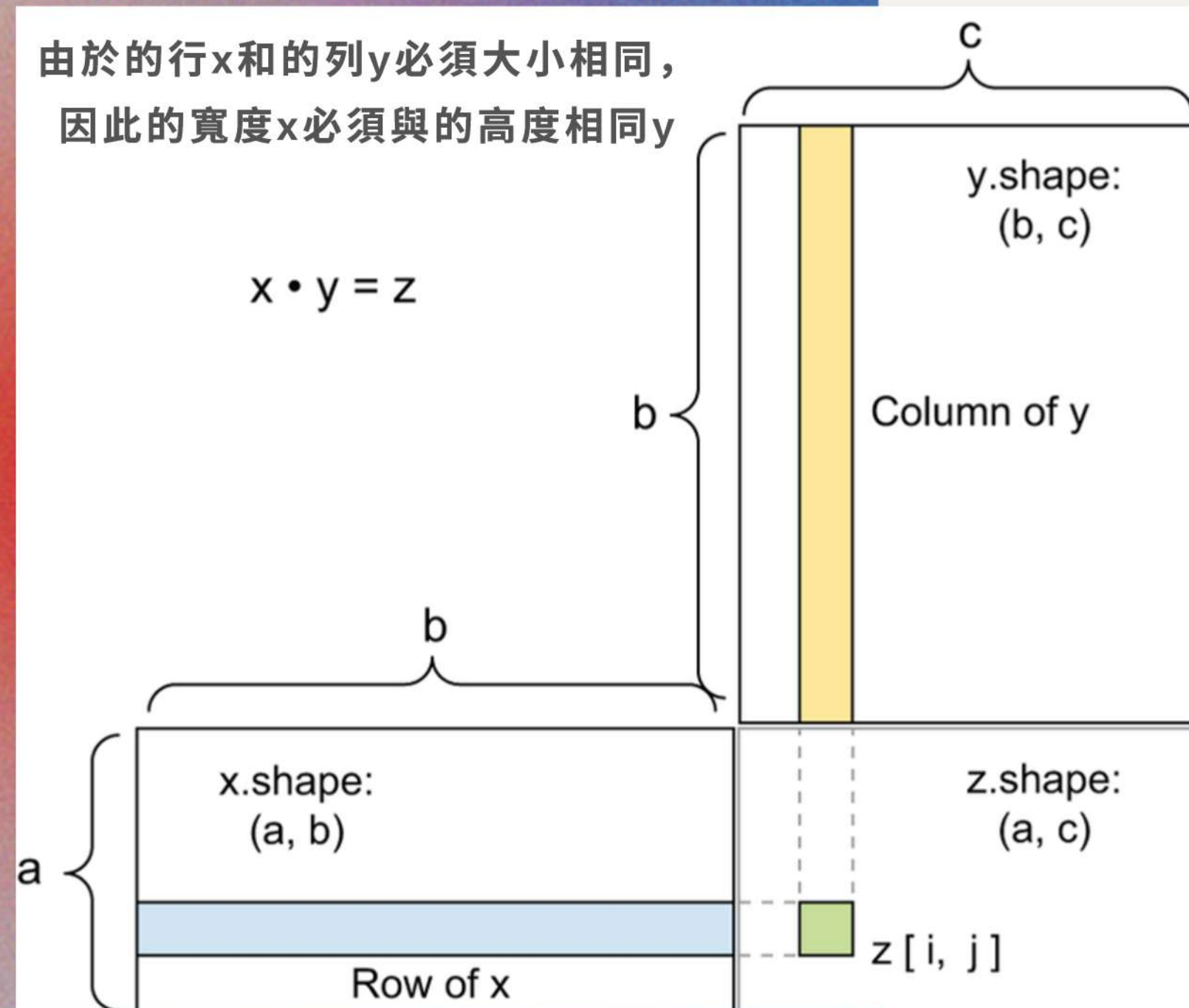
(32, 10)

```
def naive_add_matrix_and_vector(x, y):
    assert len(x.shape) == 2
    assert len(y.shape) == 1
    # 擴張規則核心: y的長度必須等於x的寬度
    assert x.shape[1] == y.shape[0]
    x = x.copy()
    for i in range(x.shape[0]):
        for j in range(x.shape[1]):
            # 這裡的y[j]被重複加到了x的每一列(row i)
            x[i, j] += y[j]
    return x
```


張量點積(Tensor Dot)

== 矩陣乘法

由於的行x和的列y必須大小相同，
因此的寬度x必須與的高度相同y



```
def naive_matrix_dot(x, y):  
    assert len(x.shape) == 2  
    assert len(y.shape) == 2  
    # 1. 檢查維度是否對齊: x的寬度(b)必須等於y的高度(b)  
    assert x.shape[1] == y.shape[0]  
    # 2. 準備輸出容器: 右下角的z矩陣, 形狀為(a, c)  
    z = np.zeros((x.shape[0], y.shape[1]))  
    for i in range(x.shape[0]):  
        for j in range(y.shape[1]):  
            # 提取運算單元(這點積的核心)  
            row_x = x[i, :]  
            column_y = y[:, j]  
            # 計算交會點的值  
            z[i, j] = naive_vector_dot(row_x, column_y)  
    return z
```

$$\begin{matrix} & x & & y & & \text{輸出結果} \\ \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} & \cdot & \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} & = & \begin{bmatrix} 22 & 28 \\ 49 & 64 \end{bmatrix} \end{matrix}$$

Dimensions and indices are shown below the matrices:

- x is 2×3 with $x.shape[0] = 2$ and $x.shape[1] = 3$.
- y is 3×2 with $y.shape[0] = 3$ and $y.shape[1] = 2$.
- The output is 2×2 .
- The condition $x.shape[1] == y.shape[0]$ is highlighted, showing that the width of x (3) equals the height of y (3).
- The index $y.shape[1]$ is shown pointing to the width of the output matrix.

張量重塑 (Reshaping)

```
>>> x = np.array([[0., 1.],  
...               [2., 3.],  
...               [4., 5.]])  
>>> x.shape  
(3, 2)
```

```
>>> x = np.zeros((300, 20))  
>>> x = np.transpose(x)  
>>> x.shape  
(20, 300)
```

```
>>> x = x.reshape((6, 1))  
>>> x  
array([[ 0.],  
       [ 1.],  
       [ 2.],  
       [ 3.],  
       [ 4.],  
       [ 5.]])  
  
>>> x = x.reshape((2, 3))  
>>> x  
array([[ 0.,  1.,  2.],  
       [ 3.,  4.,  5.]])
```

矩陣重塑中一個常見的特殊情況是轉置，矩陣意味著交換它的行和列，因此矩陣 $x[i, :]$ 變成 $x[:, i]$

想像你有 12 個樂高積木。
原本排成 3×4 的長方形。
你可以把它重組成 2×6 。
也可以排成 12×1 的一直線。
重點是積木的總數量（12個）
和內容永遠不變，變的只有
排列方式。

訓練循環 (The Training Loop)

剛剛講的範例中每個神經網路層都按如下方式轉換其輸入資料

```
output = relu(matmul(input, W) + b)
```



Keras 把這些藏在 `.fit()` 裡，這個循環就是它內部在做的事

什麼是導數與梯度? (Derivative & Gradient)

為什麼需要微積分?

我們想調整權重(W)來降低 Loss,

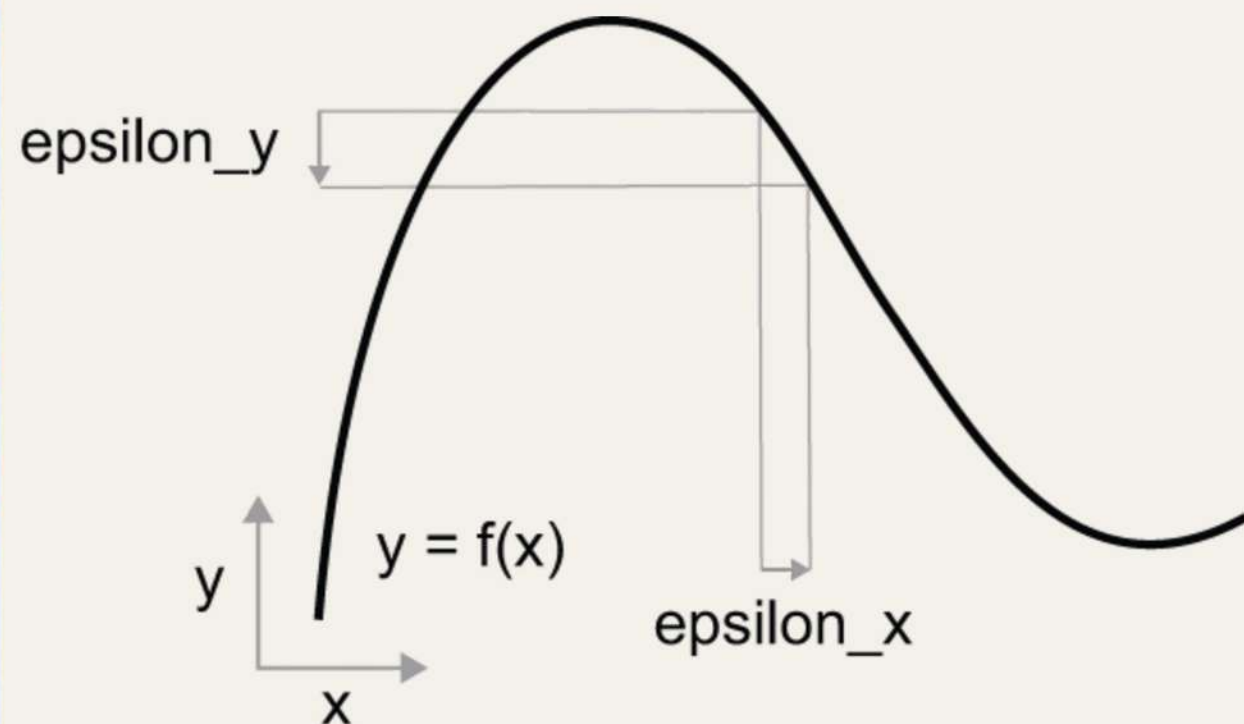
那 W 應該變大還是變小?

如果亂猜方向 -> 效率太低

參數有幾百萬個, 猜不完

如果用微積分 -> 精確算出

往哪個方向改, Loss 下降最快



導數 (Derivative): $f'(x)$

算的是 x 改變, 函數值 y 會改變多少?

針對單一變數

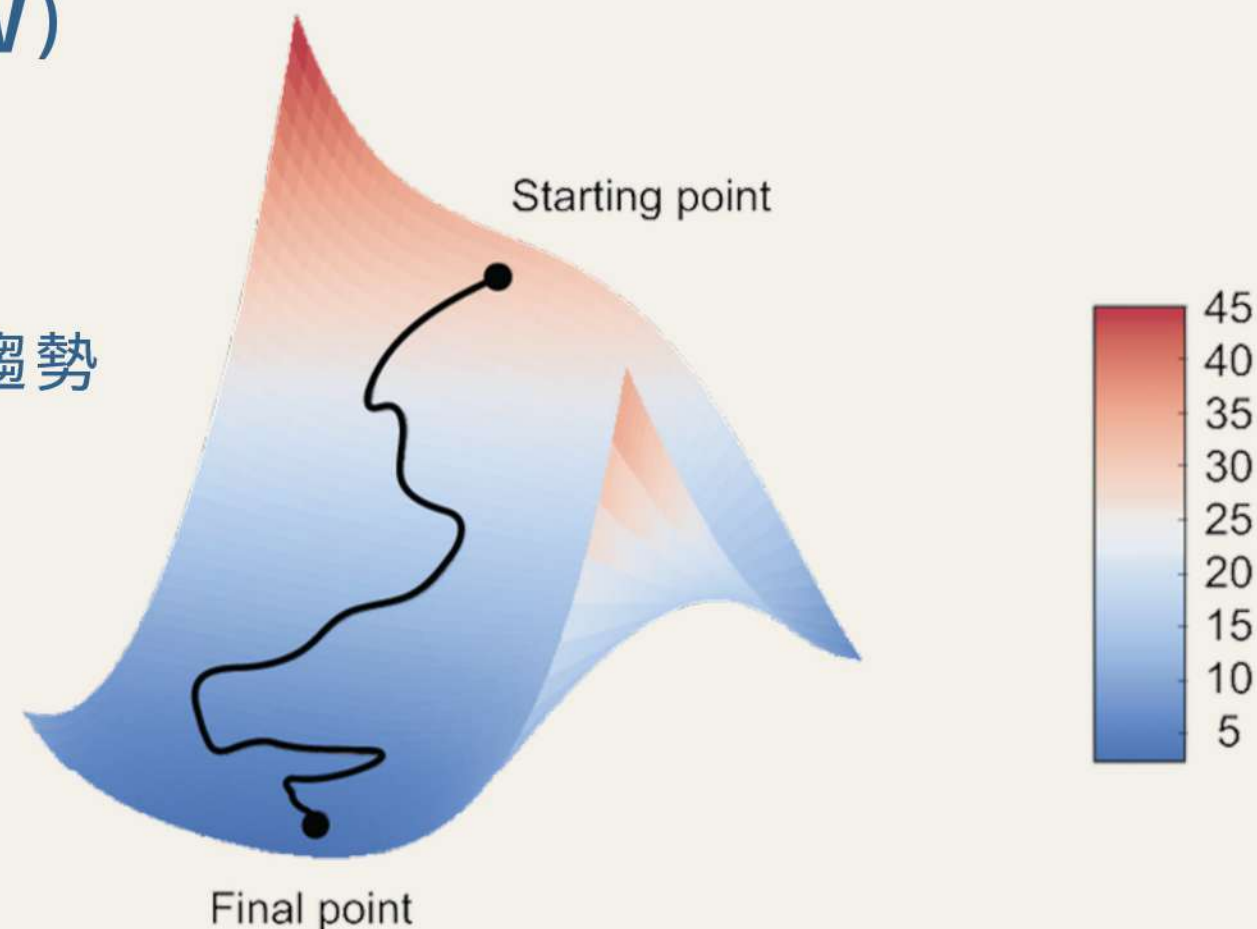
代表曲線在該點的斜率 (Slope)

梯度 (Gradient): $\nabla f(W)$

它是包含所有參數偏導數的向量

針對多變數 (張量)

代表 Loss 函數在目前位置的變化趨勢



小批次隨機梯度下降 (Mini-batch SGD)

為什麼不直接解方程式？

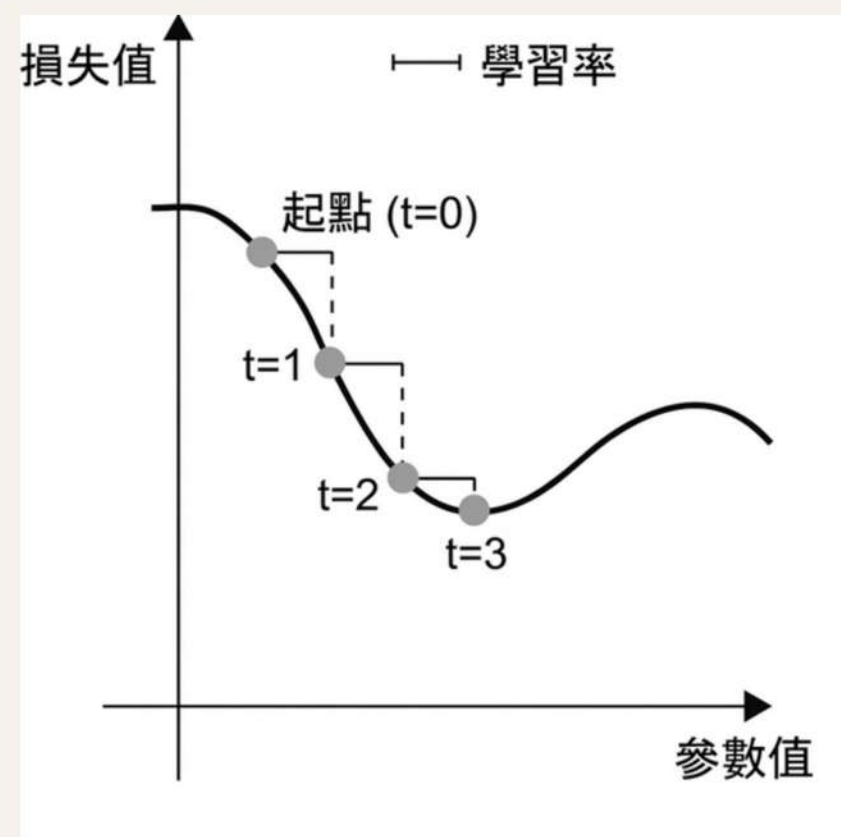
理論上我們可以解公式

$\text{grad}(f(W), W) = 0$ 找到最小值

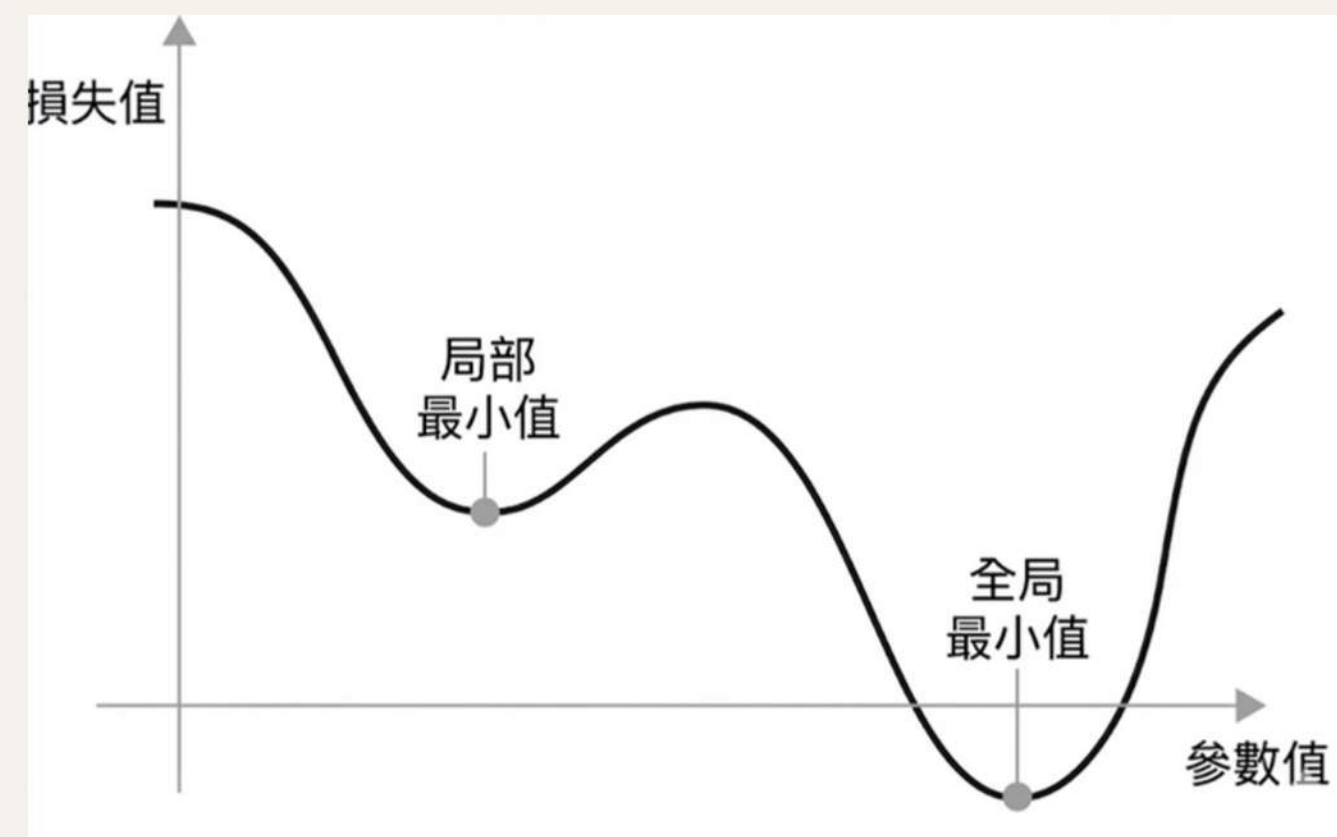
但在神經網路中，參數N可能有數百萬個，直接計算是不可行的
對此可以改用「小批次迭代」的方法，一步一步修正

SGD 的四步驟演算法

1. 抽取 (Draw): 隨機抽取一批數據樣本 x 和標籤 y_{true}
 2. 預測 (Forward): 算出預測值 y_{pred} (正向傳播)
 3. 損失 (Loss): 計算 y_{pred} 與 y_{true} 之間的差距
 4. 求導 (Backward): 計算 Loss 相對於模型參數的梯度
 5. 更新 (Update): 往梯度反方向移動一小步，以降低損失
- 公式: $W \leftarrow W - \text{learning_rate} * \text{gradient}$
- (反向傳播)



SGD 的迭代過程



梯度下降的陷阱

需要 Momentum (動量) 或調整學習率

反向傳播演算法 (Backpropagation)

我們知道 Loss，但怎麼算出第一層
權重 W 對 Loss 的貢獻？

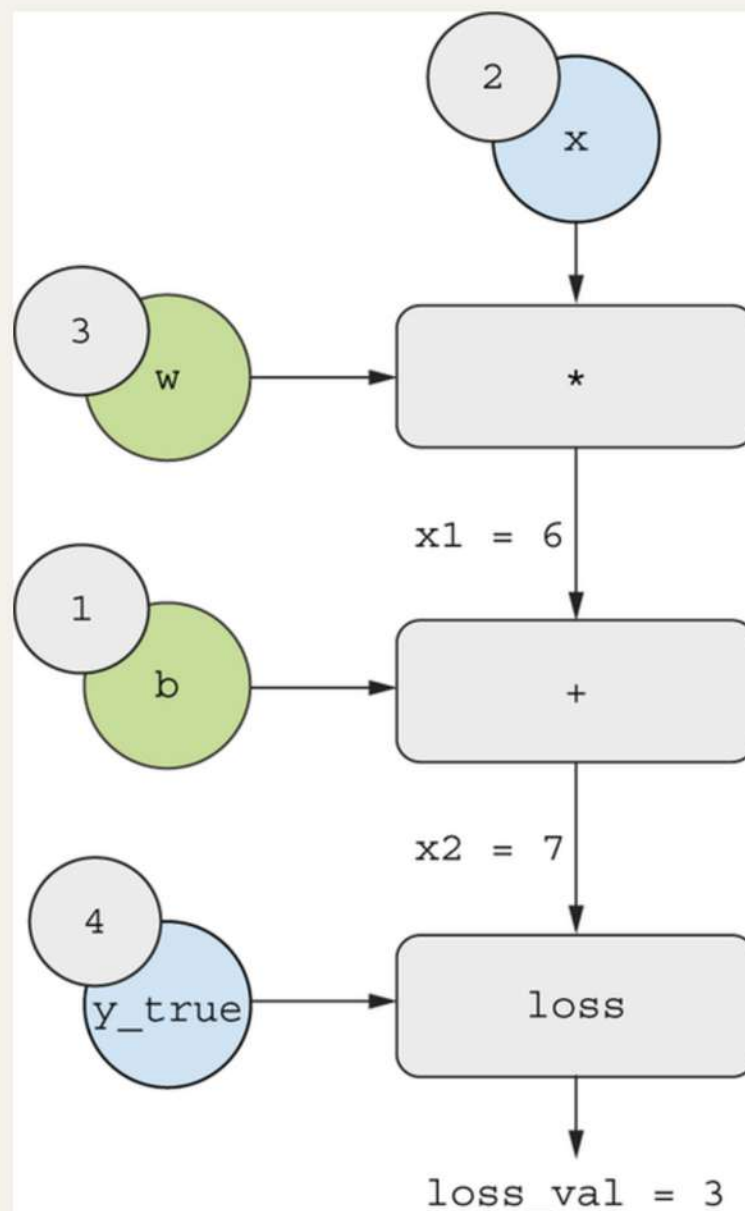
核心原理：連鎖律

$$y = f(g(x)) \Rightarrow \frac{dy}{dx} = \frac{dy}{dg} \times \frac{dg}{dx}$$

整條路徑的變化率 =
路徑上每一步變化率的乘積

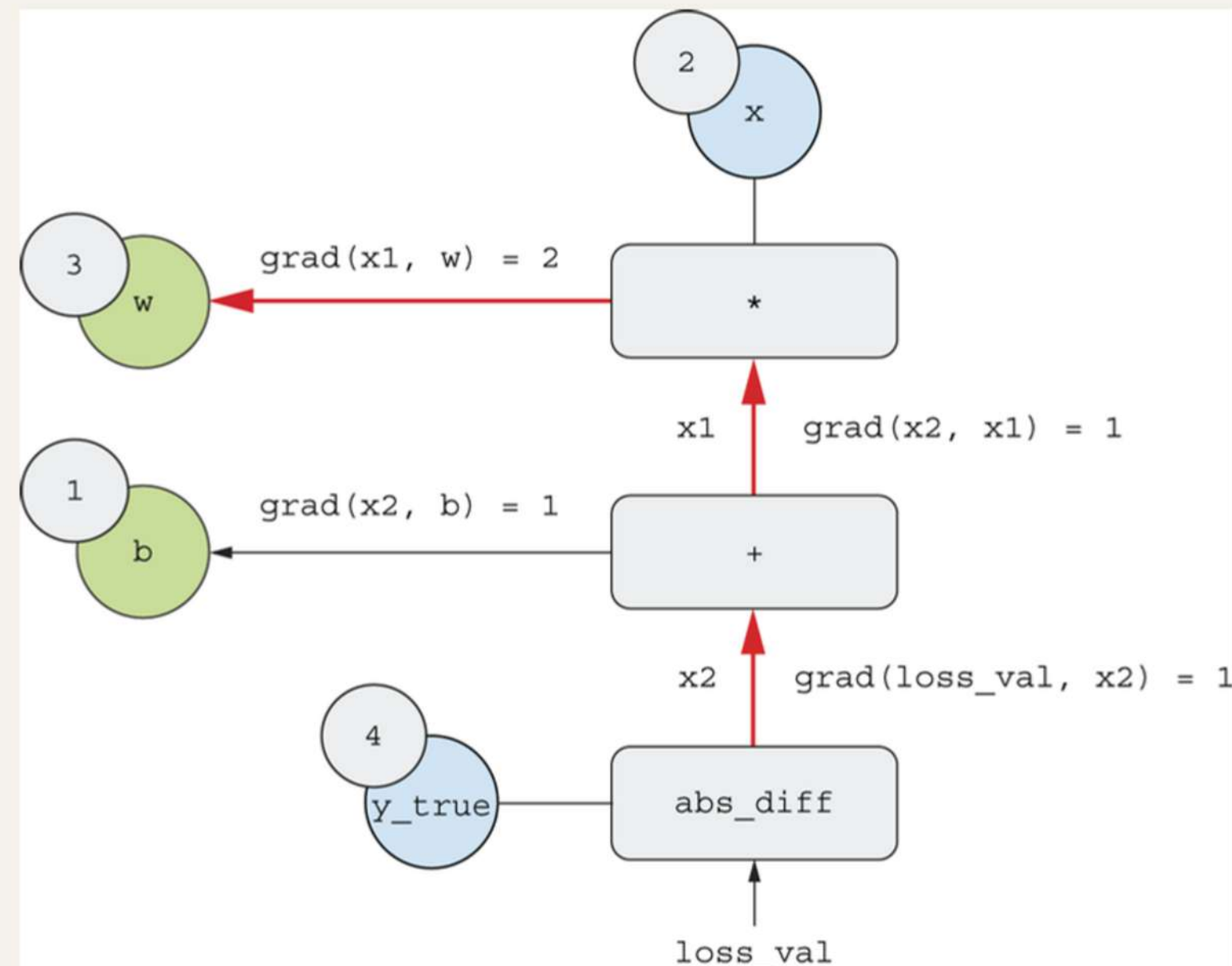
演算法運作機制

1. Forward Pass：資料從頭流到尾，算出 Loss
2. Backward Pass：從 Loss 開始，沿著計算圖的邊緣反向走回去
3. 相乘：將路徑上搜集到的導數全部乘起來，就是最終的梯度



Forward Pass

經過運算 $(x * w + b)$ 得到
預測值 $y_pred = 7$

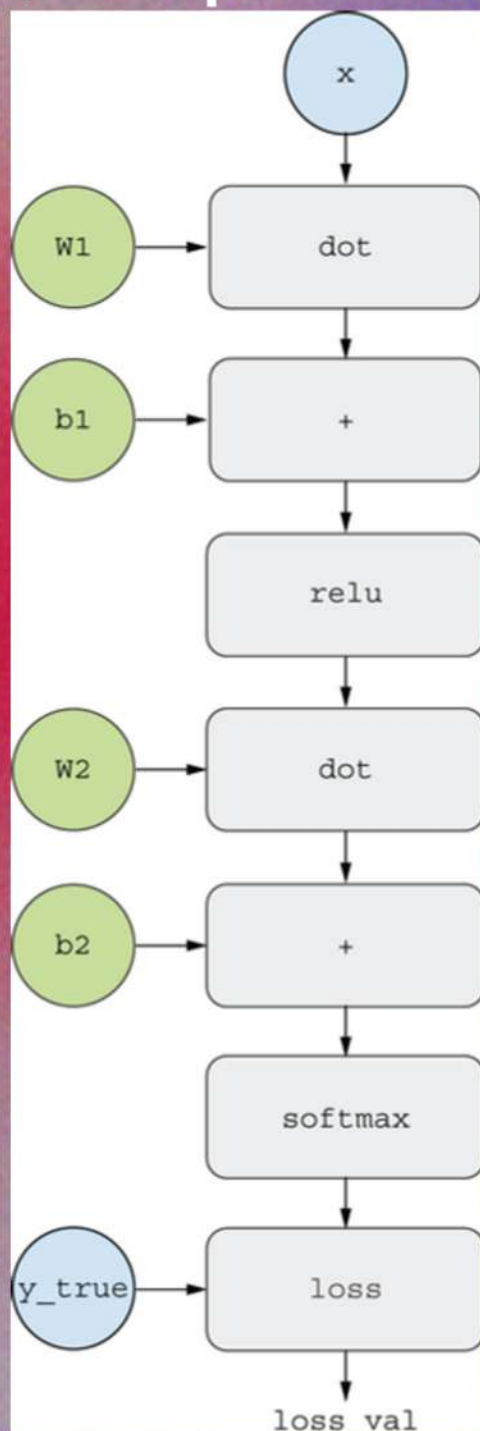


Backward Pass

我們想求 W 的梯度，就沿著紅色路徑從 Loss 走回 W
我們把路徑上的數字乘起來 $1 * 1 * 2 = 2$ ，得到 W 的梯度是 2
意思是 W 增加 1，Loss 就會增加 2

程式碼與數學的對應 (Code Mapping)

計算圖 (Computation Graph)



自動微分(TensorFlow/PyTorch)

- 框架會在背後自動建立計算圖
- GradientTape: 它會錄下做過的所有運算 (前向傳播)
- 當你需要梯度時, 它會倒帶 (反向傳播) 並自動利用連鎖律算出結果

2-4-4 連鎖導數: 反向傳播 Backpropagation 演算法

```
import tensorflow as tf
x = tf.Variable(0.)
with tf.GradientTape() as tape:
    y = 2 * x + 3
grad_of_y_wrt_x = tape.gradient(y, x)
```

```
x = tf.Variable(tf.random.uniform((2, 2)))
with tf.GradientTape() as tape:
    y = 2 * x + 3
grad_of_y_wrt_x = tape.gradient(y, x)
```

```
W = tf.Variable(tf.random.uniform((2, 2)))
b = tf.Variable(tf.zeros((2,)))
x = tf.random.uniform((2, 2))
with tf.GradientTape() as tape:
    y = tf.matmul(x, W) + b
grad_of_y_wrt_W_and_b = tape.gradient(y, [W, b])
```


回顧第一個例子

```
# 對應TensorOperations&Weights(張量運算與權重)
model = keras.Sequential([
    layers.Dense(512, activation="relu"),
    layers.Dense(10, activation="softmax")
])
```

模型架構 (The Model)

每一層都是對輸入資料進行簡單的
張量運算，運算依賴於權重(W)，
這是模型儲存知識的地方

```
# 對應損失函數(Loss) & 優化器(Optimizer)
model.compile(optimizer="rmsprop",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
```

模型編譯 (Compilation)

```
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
# 對應張量格式化 (Float32 Tensors)
train_images = train_images.reshape((60000, 28 * 28))
train_images = train_images.astype("float32") / 255
test_images = test_images.reshape((10000, 28 * 28))
test_images = test_images.astype("float32") / 255
```

資料預處理 (Input Data)

```
# 對應訓練循環(Loop:SGD & Backprop)
model.fit(train_images, train_labels, epochs=5, batch_size=128)
```

```
Epoch 1/5
469/469 ----- 6s 12ms/step - accuracy: 0.8720 - loss: 0.4356
Epoch 2/5
469/469 ----- 9s 9ms/step - accuracy: 0.9668 - loss: 0.1136
Epoch 3/5
469/469 ----- 5s 9ms/step - accuracy: 0.9786 - loss: 0.0732
Epoch 4/5
469/469 ----- 4s 9ms/step - accuracy: 0.9846 - loss: 0.0527
Epoch 5/5
469/469 ----- 5s 11ms/step - accuracy: 0.9888 - loss: 0.0385
<keras.src.callbacks.history.History at 0x7df540eae270>
```

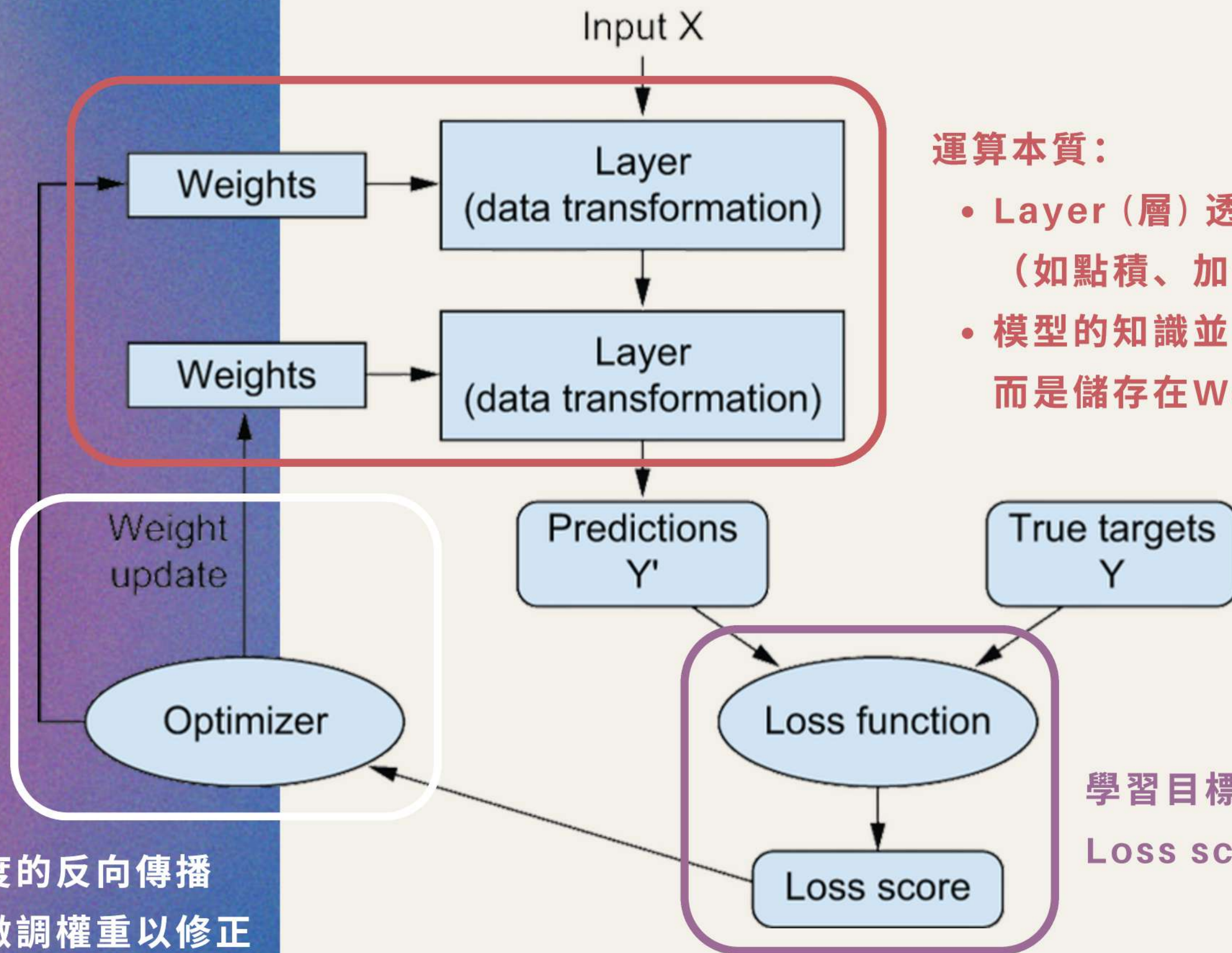
訓練循環 (Training Loop)

以 128 筆資料為一批次 (Mini-batch) 進行迭代，
利用反向傳播計算梯度，並移動權重以降低 Loss

從頭開始重新建構模型

CH2 總結 (Summary)

資料結構：所有的輸入資料 (X) 與目標 (Y) 都必須編碼成數字張量 (dtype, rank, shape)



運算本質：

- Layer (層) 透過簡單的張量運算 (如點積、加法) 來轉換資料
- 模型的知識並非寫死在程式碼中，而是儲存在Weights(權重)張量裡

優化機制：

- 利用連鎖律計算梯度的反向傳播
- 優化器根據梯度，微調權重以修正錯誤的隨機梯度下降

學習目標：就是尋找一組權重，讓 Loss score(損失分數)降到最低